Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №1 по курсу

«Операционные системы»

Группа: М8О-210БВ-25

Студент: Никитин А.А.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 17.10.26

Москва, 2026

Постановка задачи

Вариант 17.

Составить и отладить программу на языке Си, осуществляющую работу с процессами и взаимодействие между ними в одной из двух операционных систем. В результате работы программа (основной процесс) должен создать для решение задачи один или несколько дочерних процессов. Взаимодействие между процессами осуществляется через системные сигналы/события и/или каналы (pipe). Необходимо обрабатывать системные ошибки, которые могут возникнуть в результате работы.

Родительский процесс создает два дочерних процесса. Первой строкой пользователь в консоль родительского процесса вводит имя файла, которое будет использовано для открытия File с таким именем на запись для child1. Аналогично для второй строки и процесса child2. Родительский и дочерний процесс должны быть представлены разными программами.

Родительский процесс принимает от пользователя строки произвольной длины и пересылает их в pipe1 или в pipe2 в зависимости от правила фильтрации. Процесс child1 и child2 производят работу над строками. Процессы пишут результаты своей работы в стандартный вывод.

Вариант 17) Правило фильтрации: строки длины больше 10 символов отправляются в pipe2, иначе в pipe1. Дочерние процессы удаляют все гласные из строк.

Общий метод и алгоритм решения

Использованные системные вызовы:

- pid_t fork(void); – создает дочерний процесс.
- int pipe(int *fd); – создает односторонний канал связи
- int dup2(int *oldfd*, int *newfd*); – подменяет файловый дескриптор
- pid_t getpid(void); – возвращает id процесса
- int open(const char **path*, int *flags*, … /* mode_t *mode* */); – открывает файл
- int close(int *fd*); – закрывает файловый дескриптор
- ssize_t read(size_t count; int *fd*, void *buf*[*count*], size_t *count*); – читает из файлового дескриптора
- ssize_t write(size_t count; int *fd*, const void *buf*[*count*], size_t *count*); – пишет в файловый дескриптор
- ssize_t readlink(size_t bufsiz; const char *restrict *path*, char *buf*[restrict *bufsiz*], size_t *bufsiz*); – читает значение символической ссылки
- int execv(const char **path*, char *const *argv*[]); – подменяет процесс
- pid_t wait(int *_Nullable *wstatus*); – ждет звершения дочернего процесса
- void exit(int *status*); – завершает процесс

В рамках лабораторной работы была разработана программа, состоящая из двух частей клиента и сервера. Клиент создает два дочерних процесса серверных процесса и перенаправляет данные из терминала одному из серверов в зависимости от длины входных данных. Сервер обрабатывает полученные данные, удаляя гласные, и записывает полученный результат в файл.

Код программы

client.c

```
#include <stdint.h>
#include <stdbool.h>

#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>
#include <stdio.h>
#include <ctype.h>

static char SERVER_PROGRAM_NAME[] = "server";

int main(int argc, char **argv) {
	if (argc <= 2) {
		char msg[1024];
		uint32_t len = snprintf(msg, sizeof(msg), "usage: %s filename1 filename2\n", argv[0]);
		write(STDERR_FILENO, msg, len);
		exit(EXIT_SUCCESS);
	}

	char progpath[1024];
	ssize_t len = readlink("/proc/self/exe", progpath, sizeof(progpath) - 1);
	if (len == -1) {
		const char msg[] = "error: failed to read full program path\n";
		write(STDERR_FILENO, msg, sizeof(msg));
		exit(EXIT_FAILURE);
	}

	while (progpath[len] != '/') {
		--len;
	}

	progpath[len] = '\0';

	int client_to_server1[2];
	if (pipe(client_to_server1) == -1) {
		const char msg[] = "error: failed to create pipe\n";
		write(STDERR_FILENO, msg, sizeof(msg));
		exit(EXIT_FAILURE);
	}

	int server_to_client1[2];
	if (pipe(server_to_client1) == -1) {
		const char msg[] = "error: failed to create pipe\n";
		write(STDERR_FILENO, msg, sizeof(msg));
		exit(EXIT_FAILURE);
```

```
	}

	const pid_t child = fork();

	switch (child) {
	case -1: {
		const char msg[] = "error: failed to spawn new process\n";
		write(STDERR_FILENO, msg, sizeof(msg));
		exit(EXIT_FAILURE);
	} break;
	case 0: {
		pid_t pid = getpid();

		char msg[64];
		const int32_t length = snprintf(msg, sizeof(msg), "%d: I'm a child\n",
pid);
		write(STDOUT_FILENO, msg, length);

		close(client_to_server1[1]);
		close(server_to_client1[0]);

		dup2(client_to_server1[0], STDIN_FILENO);
		close(client_to_server1[0]);

		dup2(server_to_client1[1], STDOUT_FILENO);
		close(server_to_client1[1]);

		char path[1024];
		snprintf(path, sizeof(path) - 1, "%s/%s", progpath, SERVER_PROGRAM_NAME);

		char *const args[] = {SERVER_PROGRAM_NAME, argv[1], "1", NULL};

		int32_t status = execv(path, args);

		if (status == -1) {
			const char msg[] = "error: failed to exec into new exectuable
image\n";
			write(STDERR_FILENO, msg, sizeof(msg));
			exit(EXIT_FAILURE);
		}
	} break;
	default: {
		int client_to_server2[2];
		if (pipe(client_to_server2) == -1) {
			const char msg[] = "error: failed to create pipe\n";
			write(STDERR_FILENO, msg, sizeof(msg));
			exit(EXIT_FAILURE);
		}

		int server_to_client2[2];
```

```
		if (pipe(server_to_client2) == -1) {
			const char msg[] = "error: failed to create pipe\n";
			write(STDERR_FILENO, msg, sizeof(msg));
			exit(EXIT_FAILURE);
		}

		const pid_t childchild = fork();

		switch (childchild) {
		case -1: {
			const char msg[] = "error: failed to spawn new process\n";
			write(STDERR_FILENO, msg, sizeof(msg));
			exit(EXIT_FAILURE);
		} break;
		case 0: {
			pid_t pid = getpid();

			char msg[64];
			const int32_t length = snprintf(msg, sizeof(msg),
				"%d: I'm a child\n", pid);
			write(STDOUT_FILENO, msg, length);

			close(client_to_server2[1]);
			close(server_to_client2[0]);

			dup2(client_to_server2[0], STDIN_FILENO);
			close(client_to_server2[0]);

			dup2(server_to_client2[1], STDOUT_FILENO);
			close(server_to_client2[1]);

			char path[1024];
			snprintf(path, sizeof(path) - 1, "%s/%s", progpath,
SERVER_PROGRAM_NAME);

			char *const args[] = {SERVER_PROGRAM_NAME, argv[2], "2", NULL};

			int32_t status = execv(path, args);

			if (status == -1) {
				const char msg[] = "error: failed to exec into new
exectuable image\n";
				write(STDERR_FILENO, msg, sizeof(msg));
				exit(EXIT_FAILURE);
			}
		} break;
		default: {
			pid_t pid = getpid();

			char msg[128];
```

```
                    const int32_t length = snprintf(msg, sizeof(msg),
                            "%d: I'm a parent, my children has PID %d and PID %d\n",
pid, child, childchild);
                    write(STDOUT_FILENO, msg, length);

                    close(client_to_server1[0]);
                    close(server_to_client1[1]);
                    close(client_to_server2[0]);
                    close(server_to_client2[1]);

                    char buf[4096];
                    ssize_t bytes;

                    while (bytes = read(STDIN_FILENO, buf, sizeof(buf))) {
                            if (bytes < 0) {
                                    const char msg[] = "error: failed to read from
stdin\n";
                                    write(STDERR_FILENO, msg, sizeof(msg));
                                    exit(EXIT_FAILURE);
                            }
                            else if (buf[0] == '\n') {
                                    break;
                            }

                            int* client_to_server = bytes > 11 ? &client_to_server2[0] :
&client_to_server1[0];
                            int* server_to_client = bytes > 11 ? &server_to_client2[0] :
&server_to_client1[0];

                            write(client_to_server[1], buf, bytes);
                            bytes = read(server_to_client[0], buf, sizeof(buf));
                            write(STDOUT_FILENO, buf, bytes);
                    }

                    close(client_to_server1[1]);
                    close(server_to_client1[0]);
                    close(client_to_server2[1]);
                    close(server_to_client2[0]);

                    wait(NULL);
                    wait(NULL);
            } break;
            }
      } break;
      }
}
```

server.c

```
#include <stdint.h>
#include <stdbool.h>
#include <ctype.h>
```

```
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <stdio.h>

int main(int argc, char **argv) {
	if (argc <= 2) {
		char msg[1024];
		uint32_t len = snprintf(msg, sizeof(msg), "usage: %s filename number\n",
argv[0]);
		write(STDERR_FILENO, msg, len);
		exit(EXIT_SUCCESS);
	}

	char buf[4096];
	char buf1[4096];
	ssize_t bytes;
	ssize_t len;

	int32_t file = open(argv[1], O_WRONLY | O_CREAT | O_TRUNC, 0600);
	if (file == -1) {
		const char msg[] = "error: failed to open requested file\n";
		write(STDERR_FILENO, msg, sizeof(msg));
		exit(EXIT_FAILURE);
	}

	while (bytes = read(STDIN_FILENO, buf, sizeof(buf))) {
		if (bytes < 0) {
			const char msg[] = "error: failed to read from stdin\n";
			write(STDERR_FILENO, msg, sizeof(msg));
			exit(EXIT_FAILURE);
		}

		len = 0;
		for (uint32_t i = 0; i < bytes; ++i) {
			char ch = buf[i];
			char lch = tolower(buf[i]);
			if (lch == 'a' || lch == 'o' || lch == 'u' || lch == 'i' || lch ==
'e' || lch == 'y') {
				continue;
			}
			buf1[len] = ch;
			len++;
		}

		int32_t written = write(file, buf1, len);
		if (written != len) {
			const char msg[] = "error: failed to write to file\n";
			write(STDERR_FILENO, msg, sizeof(msg));
```

```
                exit(EXIT_FAILURE);
            }

            char msg[64];
            const int32_t length = snprintf(msg, sizeof(msg), "Server%s received: ",
argv[2]);
            write(STDERR_FILENO, msg, length);

            written = write(STDOUT_FILENO, buf1, len);
            if (written != len) {
                const char msg[] = "error: failed to echo\n";
                write(STDERR_FILENO, msg, sizeof(msg));
                exit(EXIT_FAILURE);
            }
        }

        close(file);
}
```

Протокол работы программы

```
$ ./client test1 test2
164213: I'm a child
164212: I'm a parent, my children has PID 164213 and PID 164214
164214: I'm a child
abcdef
Server1 received: bcdf
abcdefghijklmnop
Server2 received: bcdfghjklmnp
1234567890
Server1 received: 1234567890
12345678901
Server2 received: 12345678901
^C
$ cat < test1
bcdf
1234567890
$ cat < test2
bcdfghjklmnp
12345678901
```

Вывод

В ходе выполнения лабраторной работы были получены навыки работы с системными вызовами в языке программирования Си и навыки разработки многопроцессных программ. Также были изучены механизмы обмена данными между ними посредством каналов связи. В конечном итоге была реализована надежная система, которая полностью удовлетворяет условиям поставленного задания.